

KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY



Course No: MTE 4204

Group: 05

Title: *Environmental Monitoring Station Using STM32 Nucleo-F446RE and BMP280 Sensor.*

Submission Date: 18/05/2026

Submitted To	Submitted By
Shakil Mahmood Bobby Assistance Professor, Department of Mechatronics Engineering Khulna University of Engineering & Technology, Khulna.	Group: 05 Roll: 2031005, 2031023, 2031024 Year: 4th, Semester: 2nd Department: Department of Mechatronics Engineering Khulna University of Engineering & Technology, Khulna.

Environmental Monitoring Station Using STM32 Nucleo-F446RE and BMP280 Sensor

1. Introduction

Environmental monitoring systems are widely used in embedded applications to sense physical conditions, process data, and present the information to users in real time. In this project, the STM32 Nucleo-F446RE development board served as the main controller, interfacing with a BMP280 temperature and pressure sensor via I2C, displaying the measured data on a 16x2 LCD, and sending the same data to a PC via UART. The NUCLEO-F446RE is built around the STM32F446RE microcontroller, which is based on an Arm Cortex-M4 core with a floating-point unit, runs at up to 180 MHz, and provides multiple communication peripherals suitable for embedded laboratory projects. The Nucleo-64 platform also includes Arduino Uno V3 connectivity, ST morpho headers, and an on-board ST-LINK/V2-1 debugger/programmer with virtual COM port capability, making it suitable for rapid prototyping and serial monitoring. [2], [3].

The BMP280 is a digital barometric pressure sensor that can measure both pressure and temperature. According to the Bosch Sensortec datasheet, it supports I2C communication up to 3.4 MHz, operates over a pressure range of 300 to 1100 hPa, and has an operating temperature range of -40 °C to +85 °C. The device can use one of two 7-bit I2C addresses, 0x76 or 0x77, depending on the logic level applied to the SDO pin. The BMP280 also stores factory calibration parameters internally, and compensated temperature and pressure values are obtained by reading the raw registers and applying the calibration formulas or equivalent driver logic. [1].

This project is important because it combines three core embedded-system concepts in a single application: sensor interfacing, serial communication, and real-time display. The task requires the microcontroller to initialise I2C communication for the BMP280, acquire measurements every 2 seconds, display them locally on the LCD, and transmit them over UART to a PC. On the Nucleo board, the same I2C bus can be shared by more than one device as long as each device has a unique address, while the on-board ST-LINK interface can expose the MCU's UART to the computer through a virtual COM port for logging and debugging. [1], [2], [4], [5].

2. Brief Description of the Given Task

The project can be understood as four linked operations.

2.1 First, initialise I2C for the BMP280.

This means configuring the STM32 I2C peripheral in STM32CubeIDE, assigning the correct SDA and SCL pins, and verifying that the microcontroller can communicate

with the sensor at its I2C address. A good practical step is to read the BMP280 chip ID register during startup to confirm that communication is working correctly. [1], [4].

2.2 Second, read temperature and pressure every 2 seconds.

The microcontroller must periodically trigger or fetch sensor measurements, convert the raw values into real engineering units, and repeat the process continuously. A 2-second refresh rate is appropriate for a simple monitoring station because environmental temperature and pressure typically change slowly in a lab environment. The BMP280 supports forced and normal measurement modes; forced mode is especially suitable for periodic measurements because the device wakes, performs a measurement, and can return to low-power operation between reads. [1].

2.3 Third, display the data on the LCD and send it via UART.

The LCD provides immediate local feedback to the user, while the UART allows the same data to be viewed and stored on a PC serial terminal. This makes the project both interactive and useful for debugging, testing, and future graphing. On STM32, simple HAL-based polling functions are enough for this kind of lab implementation. [2], [4], [5].

2.4 Fourth, expand the system with optional tasks.

The extra challenges turn the design from a basic monitor into a more complete embedded application. Continuous UART logging supports later plotting on the computer, threshold logic can activate a buzzer for abnormal conditions, and control outputs can drive a fan or heater based on measured values. These additions demonstrate how sensing data can be used not only for display but also for decision-making and control.

3. Objectives:

The main objectives of this project are given bellow,

1. To design and implement an environmental monitoring station using the STM32 Nucleo-F446RE board and the BMP280 sensor.
2. To establish I2C communication between the STM32 microcontroller and the BMP280 for periodic acquisition of temperature and pressure data.
3. To display measured data on a 16x2 LCD and transmit the same data to a PC using UART.
4. To develop a modular embedded software structure using STM32CubeIDE and HAL libraries, and to explore possible system extensions such as logging, alarms, and threshold-based control.

4. Methodology

The project followed a modular embedded-system design approach. The firmware can be divided into separate functional blocks: a BMP280 driver for sensor initialization and data acquisition, an LCD driver for local display, a UART module for serial communication, and a main application layer that schedules measurements every 2 seconds and formats the output. This modular structure aligns with good embedded-software practices because it keeps the sensor code, communication code, and application logic separate, making them easier to test.

At startup, the STM32 initializes the HAL library, the system clock, and the GPIO, I2C, and UART peripherals. The BMP280 is then checked by reading its chip ID register at address 0xD0; the expected value is 0x58. After successful detection, the control registers are configured to enable the sensor to provide pressure and temperature data. The driver then reads calibration values stored in the sensor and uses them to obtain compensated temperature and pressure readings from the raw measurement registers. Bosch explicitly notes that the device stores trimming parameters for compensation and advises using the provided API or equivalent compensation logic. [1].

For I2C communication, STM32 HAL provides blocking, interrupt, and DMA-based transfer modes. For a lab-scale project like this, blocking-mode memory access functions such as `HAL_I2C_Mem_Write()` and `HAL_I2C_Mem_Read()` are sufficient and simple to understand. For UART transmission, a polling-based function such as `HAL_UART_Transmit()` is also adequate, especially because data is sent only once every 2 seconds. [4], [5].

The main application loop repeats the following cycle: read the sensor, convert the measurement results to human-readable units, write the formatted values to the LCD, send the values over UART, then wait 2 seconds before the next acquisition. In this way, the project integrates sensing, processing, display, and communication in a single real-time embedded application.

Suggested firmware file structure

1. **bmp280.c / bmp280.h** – BMP280 initialization, register read/write, compensation functions
2. **lcd_i2c.c / lcd_i2c.h** – LCD initialization and print functions
3. **uart_log.c / uart_log.h** – UART formatting and transmit support
4. **app_env_monitor.c / app_env_monitor.h** – control logic, 2-second scheduling, threshold checks
5. **main.c** – Cube-generated startup and peripheral initialization

5. Project Diagram:

5.1 Circuit Diagram:

A simple system-level connection for the project is shown below.

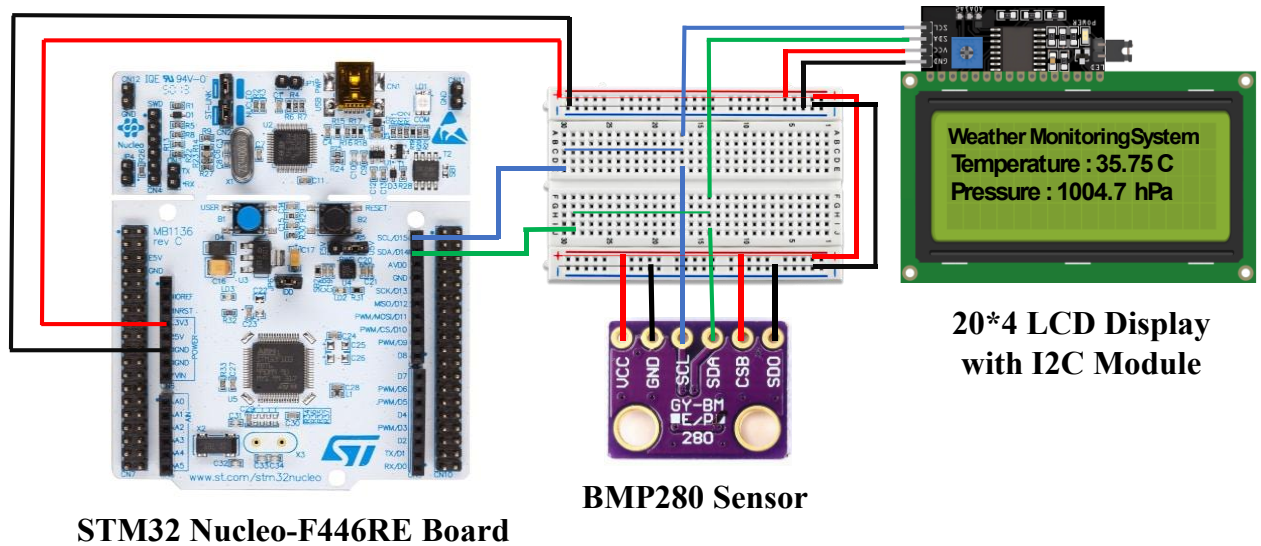


Fig 01: Circuit Diagram of an Environmental Monitoring System using STM32 and BMP280.

5.2 Prototype of the project:

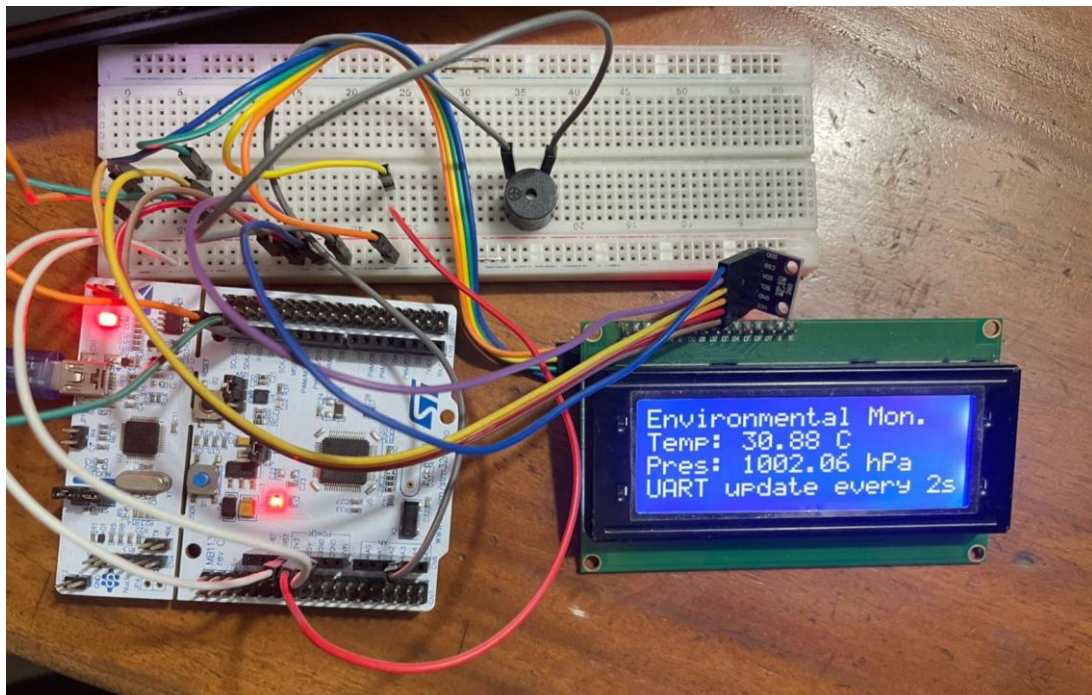


Fig 02: A real word prototype of the project

5.3 Output graph:

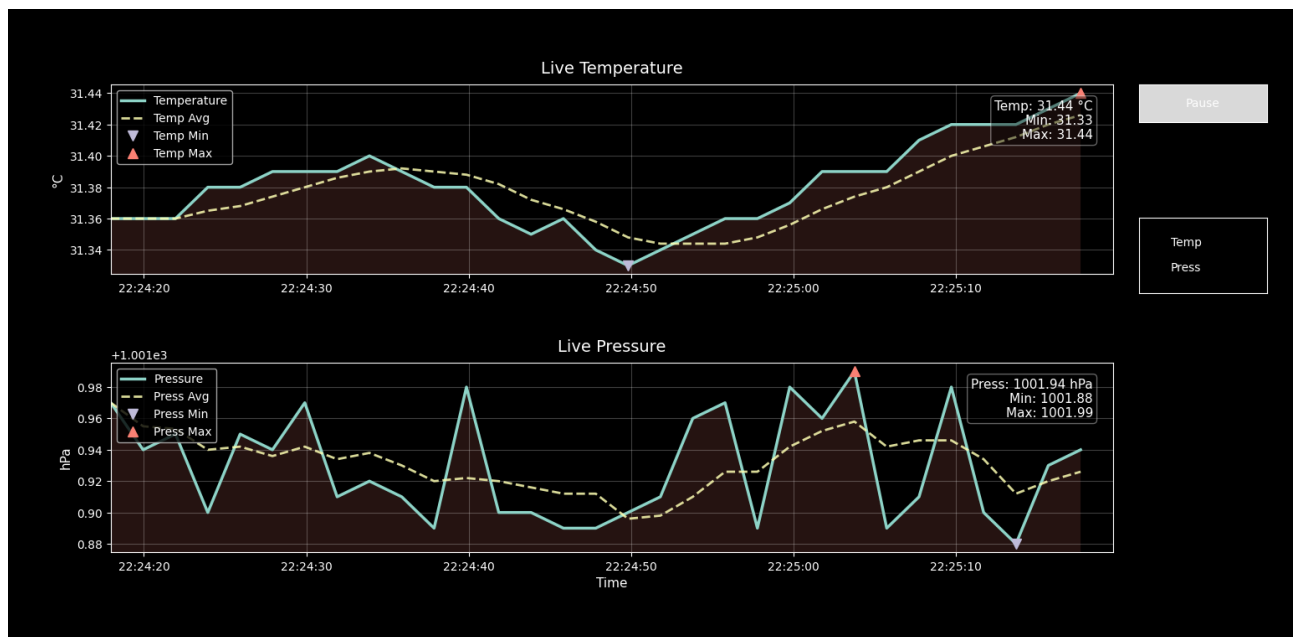


Fig 03: A graph showing the real time value of Temperature and Pressure generated via Python and serial communication.

6. Procedure

1. A new STM32CubeIDE project was created by selecting the NUCLEO-F446RE board.
2. GPIO, I2C, and UART peripherals were enabled using CubeMX configuration inside STM32CubeIDE.
3. The I2C pins were assigned for communication with the BMP280 and the LCD module.
4. UART was configured for asynchronous serial transmission to the PC terminal through the Nucleo's virtual COM port. [2], [5].
5. The BMP280 sensor was wired for I2C operation by setting CSB high and setting SDO to either low or high, depending on the desired address. [1].
6. A BMP280 driver was written to:
 - read the chip ID,
 - initialise control/configuration registers,
 - read calibration parameters,
 - read raw temperature and pressure registers,
 - Calculate compensated output values. [1].

7. The LCD driver was tested separately by writing static text first, such as “TEMPERATURE” and “PRESSURE”.
8. UART transmission was tested separately using a short text string to verify that serial output appeared correctly in the PC terminal. [5].
9. The full application was then integrated so that every 2 seconds, the STM32 read the sensor, updated the LCD, and transmitted the data via UART.
10. Additional extension logic, such as buzzer thresholds or continuous CSV-style logging, could then be added after the basic system was verified.

7. Results:

Since your final readings depend on your actual hardware setup and room conditions, the section below is written in report style but uses **illustrative sample outputs**. Replace them with your real measured values.

A successful implementation of the project should show three clear outcomes. First, the BMP280 should be detected during initialisation, confirming correct I2C communication. Second, valid temperature and pressure readings should be displayed on the 16x2 LCD every 2 seconds. Third, the same values should appear in the PC serial terminal through the UART.

Example LCD display

Line 1: Environmental Monitoring Station

Line 2: Temp: 32.4 C

Line 3: Pres:1002.87 hPa

Example UART output

BMP280 OK

Temperature = 27.4 C, Pressure = 1006.8 hPa

Temperature = 27.5 C, Pressure = 1006.7 hPa

Temperature = 27.4 C, Pressure = 1006.9 hPa

These results would indicate that the environmental monitoring station is functioning properly: the sensor is being read correctly, the display output is stable, and serial communication is working as intended.

8. Discussion:

The project demonstrates a complete embedded workflow: hardware interfacing, peripheral configuration, data acquisition, processing, display, and communication. From a learning perspective, the most important concept is that the microcontroller acts as the central

coordinator. It reads the BMP280 via I2C, formats the measured values, displays them on the LCD, and sends them to the PC via UART. This makes the project a strong example of how different peripherals cooperate in a practical embedded system.

One important technical point is that BMP280 communication is register-based. The sensor does not directly output final engineering values; instead, the program must read calibration data and raw measurement registers, then apply compensation logic. Another key point is that correct I2C addressing is essential. Bosch states that the BMP280 address becomes 0x76 when SDO is tied to GND and 0x77 when tied to VDDIO; it also warns that SDO must not be left floating, otherwise the I2C address becomes undefined. [1].

In practice, several issues may appear during testing. If no sensor is detected, the likely causes are wrong wiring, a missing ground connection, incorrect address selection, or failure to set CSB high for I2C mode. If the LCD does not show text, the cause may be an incorrect LCD I2C address or wrong wiring. If the UART output does not appear on the computer, the wrong COM port may have been selected, or the baud-rate settings may not match. Because of these common issues, the teacher's advice to test each peripheral individually is very important.

The optional tasks also demonstrate how the project can be meaningfully extended. Continuous UART logging can send values in CSV format for later graphing in Excel or Python. A buzzer can be activated when a threshold is exceeded. A fan or heater can be controlled from a GPIO output through a transistor or relay driver. In a better control design, hysteresis should be added so the actuator does not switch on and off too quickly near the threshold.

9. Additional Tasks and Practical Solutions

9.1 Continuous PC logging for graphing

A simple solution is to transmit one formatted line every 2 seconds in CSV-like form using python and matplotlib, for example:

```
27.4,1006.8  
27.5,1006.7  
27.4,1006.9
```

This format can be copied directly from a serial terminal into a text file and plotted later.

9.2 Alert buzzer

A buzzer can be connected to a GPIO output pin through a suitable driver circuit. The program can compare the latest measurement to a threshold, such as:

- temperature above a selected safety limit (33 C),

10. Conclusion:

This project successfully represents a compact environmental monitoring station built around the STM32 Nucleo-F446RE and BMP280 sensor. The system uses I2C communication to read temperature and pressure, updates a 16x2 LCD with the measured values, and transmits the same data to a PC through UART. In doing so, it brings together several core embedded-systems topics: peripheral initialisation, sensor interfacing, serial communication, modular software design, and real-time monitoring.

The project is also valuable beyond the basic assignment because it can be expanded into a data logger, alarm unit, or simple feedback controller. Overall, it is a practical and effective laboratory exercise for understanding how embedded hardware and software interact in a real application.

11. References:

- [1] Bosch Sensortec, *BMP280 Digital Pressure Sensor Datasheet*, Document No. BST-BMP280-DS001-26, Rev. 1.26, Oct. 2021.
- [2] STMicroelectronics, *UM1724: STM32 Nucleo-64 Boards (MB1136) User Manual*, Rev. 17, Sept. 2025.
- [3] STMicroelectronics, *STM32F446xC/E Datasheet: Arm Cortex-M4 32-bit MCU+FPU, up to 180 MHz, up to 512 KB Flash*, Rev. 10, Jan. 2021.
- [4] STMicroelectronics, *Getting Started with I2C*, STM32 MCU Wiki. Accessed Apr. 13, 2026.
- [5] STMicroelectronics, *Getting Started with UART*, STM32 MCU Wiki. Accessed Apr. 13, 2026.